Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2026-27

Class:	**TE-AI&DS**	**Semester:**	**V**
Course Code:	**2015111**	**Course Name:**	**Statistics for Machine Learning and Data Science Lab**

Name of Student:	**Divya S. Konar**
Roll No. :	**20**
Experiment No.:	**7**
Title of the Experiment:	**Build and Evaluate Classification Models using Supervised Learning Algorithms**
Date of Performance:	**02/09/26**
Date of Submission:	**09/09/26**

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : Dr. Tatwadarshi P.N.

Signature :

Date :

Aim: To build supervised classification models for predicting diabetes outcomes using the Pima Indians Diabetes Dataset and evaluate their performance using appropriate classification metrics.

Objectives

After completing this experiment, the student will be able to:

1. Build supervised classification models to predict the diabetes outcome of patients.
2. Evaluate and interpret classification model performance using appropriate evaluation metrics.

Tools Required

- Python 3.x
- Google Colab / Jupyter Notebook
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn

Dataset Details

Dataset: Pima Indians Diabetes Dataset

The dataset contains medical information for **768 female patients**.

The target variable is:

- Outcome = 0: Non-diabetic
- Outcome = 1: Diabetic

Procedure

1. Load the Pima Indians Diabetes Dataset.

2. Separate the input features and target variable.
3. Identify and handle invalid or missing values.
4. Divide the dataset into training and testing sets using a stratified split.
5. Apply suitable preprocessing, such as feature scaling where required.
6. Build one or more classification models, such as:
    - Logistic Regression
    - k-Nearest Neighbors
    - Decision Tree
7. Train the selected model(s) using the training data.
8. Generate predictions for the test data.
9. Construct the confusion matrix.
10. Calculate classification performance metrics.
11. Compare the obtained results and interpret the model performance.

Description of the Experiment

Classification is a supervised learning technique used to assign observations to predefined categories.

In this experiment, the objective is to classify a patient as: Patient Health Data, Classification Model gives Non-Diabetic and Diabetic.

The experiment follows: Dataset, Feature and Target Selection, Data Preprocessing, Train-Test Split, Classification Model, Prediction, and Performance Evaluation

Detailed Description of Statistical Methods

Logistic Regression

Logistic regression predicts the probability of a binary outcome.

The logistic function is:

$$P(Y = 1) = \frac{1}{1+e^{-z}}$$

where:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p$$

The predicted probability is converted into a class using a decision threshold.

Confusion Matrix

A confusion matrix contains:

	Predicted Negative	Predicted Positive
Actual Negative	True Negative (TN)	False Positive (FP)
Actual Positive	False Negative (FN)	True Positive (TP)

Accuracy

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

Precision

$$Precision = \frac{TP}{TP+FP}$$

Precision indicates how many predicted positive cases were actually positive.

Recall

$$Recall = \frac{TP}{TP+FN}$$

Recall indicates how many actual positive cases were correctly identified.

F1-Score

$$F1 = 2\left(\frac{Precision \times Recall}{Precision+Recall}\right)$$

The F1-score balances precision and recall.

ROC-AUC

The ROC curve represents the relationship between:

- True Positive Rate
- False Positive Rate

The Area Under the Curve (AUC) provides an overall measure of the model's ability to distinguish between the two classes.

Output :

[1] ✓ 2s

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix
)

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

[3] ✓ 0s

```
url = "https://raw.githubusercontent.com/npradaschnor/Pima-Indians-Diabetes-Dataset/master/diabetes.csv"

df = pd.read_csv(url)

print(df.head())
print("\nDataset Shape:", df.shape)
```

[3]
```
df = pd.read_csv(url)

print(df.head())
print("\nDataset Shape:", df.shape)
```

```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1

Dataset Shape: (768, 9)
```

[4]
```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
None
```

[5]
```
print(df.isnull().sum())
```

```
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

[6]
```
cols = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

df[cols] = df[cols].replace(0, np.nan)

print(df.isnull().sum())
```

```
Pregnancies                 0
Glucose                     5
BloodPressure              35
SkinThickness             227
Insulin                   374
BMI                        11
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

[7]
```
df[cols] = df[cols].fillna(df[cols].median())

print("Missing values after preprocessing:")
print(df.isnull().sum())
```

```
Missing values after preprocessing:
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

[8]
```
X = df.drop('Outcome', axis=1)
y = df['Outcome']

print("Features:")
print(X.columns)

print("\nTarget:")
print(y.name)
```

```
Features:
Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
       'BMI', 'DiabetesPedigreeFunction', 'Age'],
      dtype='object')

Target:
Outcome
```

[9]
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
Training samples: 614
Testing samples: 154
```

[10]
```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Feature scaling completed!")
```

```
Feature scaling completed!
```

[11]
```
log_model = LogisticRegression(max_iter=1000)

log_model.fit(X_train_scaled, y_train)

y_pred_log = log_model.predict(X_test_scaled)
y_prob_log = log_model.predict_proba(X_test_scaled)[:, 1]

print("Logistic Regression completed!")
```

```
Logistic Regression completed!
```

```
[12]
✓ 0s
print("Logistic Regression Results")
print("--------------------------")

print("Accuracy :", accuracy_score(y_test, y_pred_log))
print("Precision:", precision_score(y_test, y_pred_log))
print("Recall   :", recall_score(y_test, y_pred_log))
print("F1-Score :", f1_score(y_test, y_pred_log))
print("ROC-AUC  :", roc_auc_score(y_test, y_prob_log))
```

```
Logistic Regression Results
--------------------------
Accuracy : 0.7077922077922078
Precision: 0.6
Recall   : 0.5
F1-Score : 0.5454545454545454
ROC-AUC  : 0.812962962962963
```

```
[13]
✓ 0s
cm_log = confusion_matrix(y_test, y_pred_log)

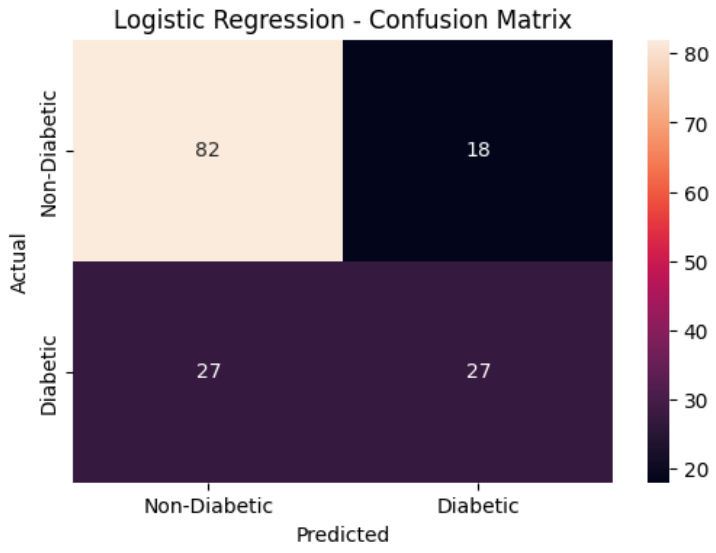
print(cm_log)

plt.figure(figsize=(6,4))

sns.heatmap(
    cm_log,
    annot=True,
    fmt='d',
    xticklabels=['Non-Diabetic', 'Diabetic'],
    yticklabels=['Non-Diabetic', 'Diabetic']
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Logistic Regression - Confusion Matrix")
plt.show()
```

```
[[82 18]
 [27 27]]
```



[14] ✓ 0s

```
knn_model = KNeighborsClassifier(n_neighbors=5)

knn_model.fit(X_train_scaled, y_train)

y_pred_knn = knn_model.predict(X_test_scaled)
y_prob_knn = knn_model.predict_proba(X_test_scaled)[:, 1]

print("KNN completed!")
```

```
KNN completed!
```

[15] ✓ 0s

```
print("KNN Results")
print("-----------")

print("Accuracy :", accuracy_score(y_test, y_pred_knn))
print("Precision:", precision_score(y_test, y_pred_knn))
print("Recall   :", recall_score(y_test, y_pred_knn))
print("F1-Score :", f1_score(y_test, y_pred_knn))
print("ROC-AUC  :", roc_auc_score(y_test, y_prob_knn))
```

```
KNN Results
-----------
Accuracy : 0.7532467532467533
Precision: 0.66
Recall   : 0.6111111111111112
F1-Score : 0.6346153846153846
ROC-AUC  : 0.7886111111111112
```

```
cm_knn = confusion_matrix(y_test, y_pred_knn)

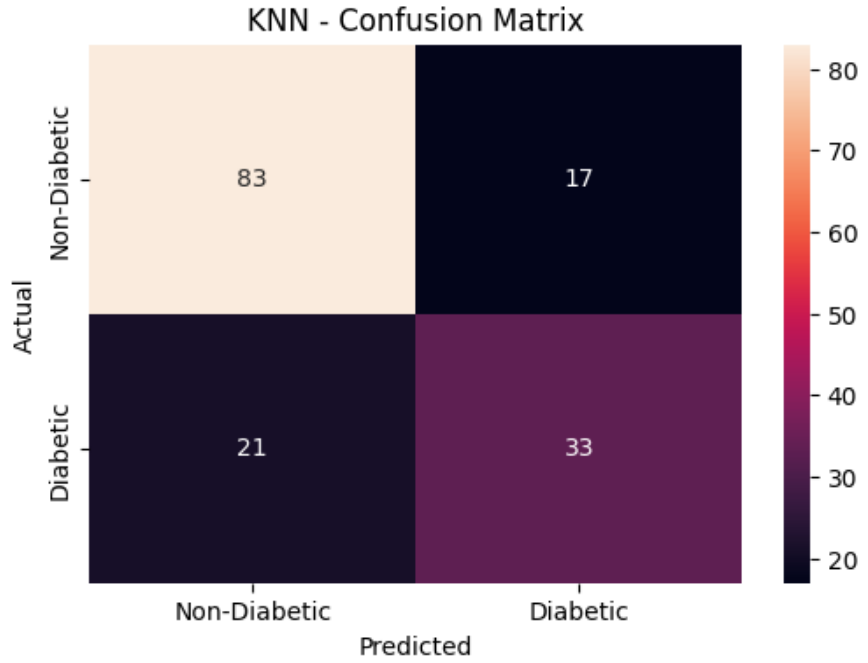
print(cm_knn)

plt.figure(figsize=(6,4))

sns.heatmap(
    cm_knn,
    annot=True,
    fmt='d',
    xticklabels=['Non-Diabetic', 'Diabetic'],
    yticklabels=['Non-Diabetic', 'Diabetic']
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("KNN - Confusion Matrix")
plt.show()
```

```
[[83 17]
 [21 33]]
```



[17] ✓ 0s

```
tree_model.fit(X_train, y_train)

y_pred_tree = tree_model.predict(X_test)
y_prob_tree = tree_model.predict_proba(X_test)[:, 1]

print("Decision Tree completed!")
```

```
Decision Tree completed!
```

[18] ✓ 0s

```
print("Decision Tree Results")
print("----------------------")

print("Accuracy :", accuracy_score(y_test, y_pred_tree))
print("Precision:", precision_score(y_test, y_pred_tree))
print("Recall   :", recall_score(y_test, y_pred_tree))
print("F1-Score :", f1_score(y_test, y_pred_tree))
print("ROC-AUC  :", roc_auc_score(y_test, y_prob_tree))
```

```
Decision Tree Results
----------------------
Accuracy : 0.6818181818181818
Precision: 0.5531914893617021
Recall   : 0.48148148148148145
F1-Score : 0.5148514851485149
ROC-AUC  : 0.6357407407407407
```

[19]
✓ 0s

```
cm_tree = confusion_matrix(y_test, y_pred_tree)

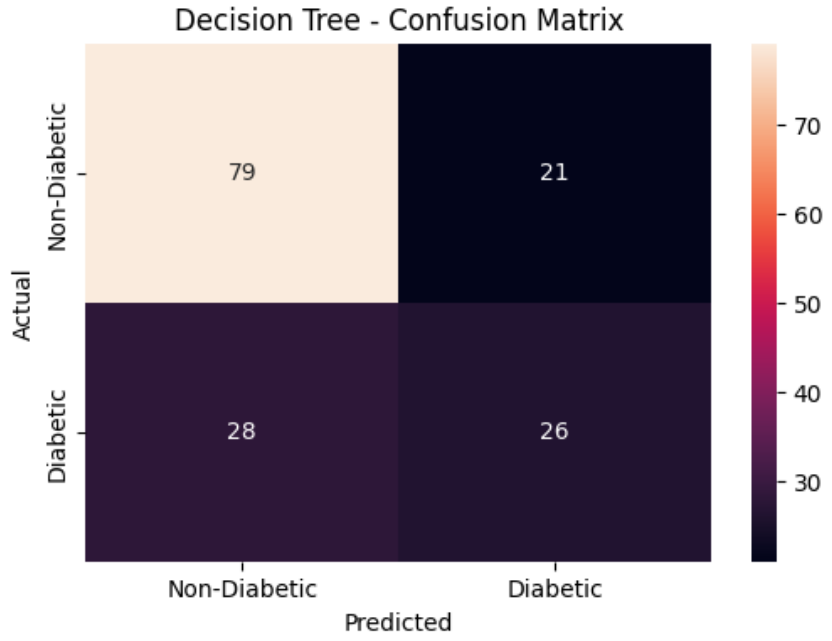
print(cm_tree)

plt.figure(figsize=(6,4))

sns.heatmap(
    cm_tree,
    annot=True,
    fmt='d',
    xticklabels=['Non-Diabetic', 'Diabetic'],
    yticklabels=['Non-Diabetic', 'Diabetic']
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Decision Tree - Confusion Matrix")
plt.show()
```

```
[[79 21]
 [28 26]]
```



[20] ✓ 0s

```
results = pd.DataFrame({
    'Model': [
        'Logistic Regression',
        'KNN',
        'Decision Tree'
    ],
    'Accuracy': [
        accuracy_score(y_test, y_pred_log),
        accuracy_score(y_test, y_pred_knn),
        accuracy_score(y_test, y_pred_tree)
    ],
    'Precision': [
        precision_score(y_test, y_pred_log),
        precision_score(y_test, y_pred_knn),
        precision_score(y_test, y_pred_tree)
    ],
    'Recall': [
        recall_score(y_test, y_pred_log),
        recall_score(y_test, y_pred_knn),
        recall_score(y_test, y_pred_tree)
    ],
    'F1-Score': [
        f1_score(y_test, y_pred_log),
        f1_score(y_test, y_pred_knn),
        f1_score(y_test, y_pred_tree)
    ],
    'ROC-AUC': [
        roc_auc_score(y_test, y_prob_log),
        roc_auc_score(y_test, y_prob_knn),
        roc_auc_score(y_test, y_prob_tree)
    ]
})

print(results)
```

```
                 Model  Accuracy  Precision    Recall  F1-Score   ROC-AUC
0  Logistic Regression  0.707792   0.600000  0.500000  0.545455  0.812963
1                  KNN  0.753247   0.660000  0.611111  0.634615  0.788611
2        Decision Tree  0.681818   0.553191  0.481481  0.514851  0.635741
```

Conclusion

Supervised classification models were developed to predict diabetes outcomes using the Pima Indians Diabetes Dataset. The models were evaluated using the confusion matrix, accuracy, precision, recall, F1-score, and ROC-AUC. The experiment demonstrated that different evaluation metrics provide different perspectives on classification performance.

Questions to be Answered by Students

1. Which classification model was implemented, and what were its accuracy, precision, recall, and F1-score?

ANS : Logistic Regression was implemented along with KNN and Decision Tree for comparison. The exact accuracy, precision, recall, and F1-score should be taken from the final Colab output, because they depend on your run.

2. How many true positives and false negatives were obtained from the confusion matrix?

ANS : These values are obtained from the Logistic Regression confusion matrix:

- **True Positive (TP):** [[82 18]
- **False Negative (FN):** [27 27]]

3. Which metric is more important for diabetes prediction: precision or recall? Justify your answer.

ANS : Recall is more important because it measures how many actual diabetic patients are correctly identified. A high recall helps reduce false negatives, where a diabetic patient is incorrectly predicted as non-diabetic.

4. What was the ROC-AUC value, and what does it indicate about the model's performance?

ANS : The ROC-AUC value is shown in your Colab output. It indicates how well the model distinguishes between diabetic and non-diabetic patients. A value closer to 1 indicates better discrimination, while 0.5 indicates performance similar to random guessing.

5. State **two limitations** of the developed classification model.

ANS : i) The dataset is relatively small and may not represent all patient populations.

ii) Replacing zero values with median values may affect the original data distribution and model performance.